

Memoria Técnica — MusicPlayer

1. Introducción

1.1 Descripción del proyecto

MusicPlayer es una plataforma web de reproducción y gestión de música desarrollada como Trabajo de Fin de Grado (TFG). Permite a los usuarios escuchar música, gestionar listas de reproducción, seguir artistas y descubrir nuevo contenido. El sistema soporta tres roles diferenciados: administrador, editor y oyente.

1.2 Objetivos

- Desarrollar una aplicación web completa (frontend + backend) con arquitectura cliente-servidor.
- Implementar un sistema de autenticación basado en tokens JWT con control de acceso por roles.
- Crear una interfaz de usuario moderna e intuitiva inspirada en plataformas de streaming actuales.
- Integrar una API externa para mostrar letras de canciones en tiempo real.
- Documentar el sistema completo para facilitar su mantenimiento y evolución.

1.3 Alcance

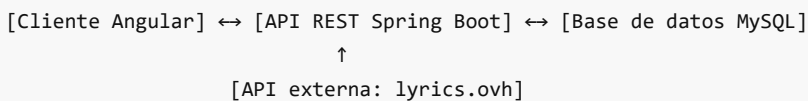
El sistema cubre:

- Registro, autenticación y gestión de usuarios.
- Catálogo de artistas, álbumes, géneros y canciones con operaciones CRUD.
- Sistema de listas de reproducción y gestión de canciones favoritas.
- Historial de reproducciones por usuario.
- Dashboards diferenciados según el rol del usuario.
- Obtención de letras de canciones mediante API externa.

2. Arquitectura del sistema

2.1 Visión general

La aplicación sigue una arquitectura de tres capas desacopladas:



El frontend Angular consume la API REST del backend mediante peticiones HTTP autenticadas con JWT. El backend gestiona la lógica de negocio, la persistencia y la seguridad. Para las letras de canciones, el frontend consulta directamente la API pública `lyrics.ovh`.

2.2 Framework base: JHipster 9.0.0

Se eligió JHipster como generador de código base por los siguientes motivos:

- Genera automáticamente la estructura inicial del proyecto con las mejores prácticas de Spring Boot y Angular.
- Incluye configuración de seguridad JWT preintegrada.
- Proporciona herramientas de migración de base de datos (Liquibase) y mapeo de entidades (MapStruct).
- Reduce el tiempo dedicado a la configuración inicial y permite centrarse en la lógica de negocio y el diseño de interfaz.

2.3 Capa frontend

Construida con **Angular 21** en modo standalone. Se comunica con el backend mediante `HttpClient` y gestiona el estado de sesión a través de JWT almacenado en `localStorage`.

Estructura principal:

```
src/main/webapp/app/
├─ app.config.ts      – Configuración de providers (HttpClient, Router, i18n)
├─ app.routes.ts      – Definición de rutas con guards
├─ core/              – Servicios transversales (auth, interceptores, config)
├─ layouts/           – Componentes de maquetación (navbar, sidebar, player-bar)
├─ home/              – Dashboards por rol (admin, editor, usuario)
├─ entities/          – Módulos CRUD de entidades (song, album, artist, etc.)
├─ account/           – Gestión de cuenta de usuario
├─ login/             – Página de autenticación
└─ shared/            – Componentes y utilidades compartidas
```

2.4 Capa backend

Construida con **Spring Boot 4.0.3**. Expone una API REST protegida por JWT. El acceso a cada endpoint está controlado por roles mediante anotaciones de Spring Security (`@PreAuthorize`).

2.5 Capa de datos

Base de datos **MySQL 8** en producción. Las migraciones se gestionan con **Liquibase**, lo que garantiza trazabilidad y reproducibilidad del esquema. Para el entorno de desarrollo se utiliza una base de datos **H2** en memoria.

3. Tecnologías del frontend

3.1 Angular 21

Se eligió Angular 21 por:

- **Tipado estático fuerte** con TypeScript, que reduce errores en tiempo de ejecución.
- **Arquitectura basada en componentes standalone**, que simplifica la estructura del proyecto eliminando la necesidad de módulos NgModule.
- **Sistema de señales reactivas** (`signal` , `computed`) para la gestión de estado local sin necesidad de librerías externas.
- **CLI potente** que facilita la generación de código, construcción y pruebas.

3.2 Bootstrap 5 + SCSS

Se empleó Bootstrap 5 para la maquetación responsive y los componentes UI base. Las personalizaciones visuales se realizaron mediante variables SCSS, lo que permite modificar el tema global desde un único punto sin alterar el código de Bootstrap.

3.3 Font Awesome 7

Iconografía vectorial escalable integrada mediante `@fortawesome/angular-fontawesome`. Los iconos se registran globalmente en `app.ts` mediante la librería de iconos, evitando importaciones duplicadas.

3.4 RxJS 7

Utilizado para la gestión de flujos asíncronos (peticiones HTTP, eventos de estado). Se usa principalmente en los servicios de entidades para transformar y manejar respuestas del backend.

3.5 ngx-translate

Internacionalización (i18n) del frontend con soporte para español e inglés. Las traducciones se definen en archivos JSON bajo `src/main/webapp/i18n/`.

4. Diseño UI/UX

4.1 Tema oscuro estilo Spotify

La decisión de adoptar un diseño oscuro similar a Spotify responde a criterios de UX específicos para aplicaciones de música:

- Los fondos oscuros reducen la fatiga visual en sesiones largas de escucha.
- El contraste con portadas de álbumes (generalmente coloridas) destaca el contenido multimedia.
- La familiaridad del patrón de diseño reduce la curva de aprendizaje para el usuario.

Paleta principal:

Token	Color	Uso
\$bg-primary	#0f172a	Fondo general
\$bg-secondary	#1e293b	Cards, sidebar
\$accent	#38bdf8	Elementos activos, hover
\$text-primary	#ffffff	Texto principal
\$text-secondary	#b3b3b3	Texto secundario

4.2 Dashboards diferenciados por rol

Se implementaron tres dashboards distintos que se muestran automáticamente según el rol del usuario autenticado:

Rol	Dashboard	Contenido principal
ROLE_ADMIN	dashboard-admin	Gestión de usuarios, métricas del sistema, accesos directos a entidades
ROLE_EDITOR / ROLE_ARTIST	dashboard-editor	Catálogo musical, gestión de álbumes y canciones propias
ROLE_USER	dashboard-user	Listas de reproducción, canciones favoritas, historial

La redirección al dashboard correcto se realiza en el componente `HomeComponent` mediante comprobación de roles con el servicio `AccountService`.

4.3 Barra lateral (Sidebar)

La sidebar ofrece navegación rápida y varía su contenido según el rol. Incluye:

- Enlace al dashboard de inicio.

- Accesos a entidades relevantes para el rol.
- Estado contraído/expandido gestionado por `SidebarService` .

4.4 Barra del reproductor (Player Bar)

Barra de reproducción fija en la parte inferior de la pantalla, siempre visible. Controles: reproducir/pausar, anterior, siguiente, aleatorio, repetir, volumen y mute. Implementada como componente standalone con señales Angular para el estado interno.

4.5 Panel de letras

Integrado en la barra del reproductor. Al pulsar el botón de letras, se despliega un panel encima de la barra que muestra la letra de la canción en reproducción, obtenida en tiempo real desde la API `lyrics.ovh` . Incluye indicador de carga y mensaje de error si no se encuentran letras.

5. Seguridad del frontend

5.1 Autenticación JWT

Flujo de autenticación:

1. El usuario envía credenciales al endpoint `/api/authenticate` .
2. El backend valida y devuelve un token JWT.
3. El frontend almacena el token y lo incluye en todas las peticiones mediante el interceptor `authInterceptor` .
4. Al expirar el token, `authExpiredInterceptor` redirige automáticamente a la página de login.

5.2 Guards de rutas

Rutas protegidas con `AuthGuard` que verifica la existencia y validez del token antes de permitir la navegación. Las rutas de administración requieren el rol `ROLE_ADMIN` .

5.3 Directivas de autorización por rol

Los elementos de la UI que solo deben mostrarse a ciertos roles se controlan mediante comprobaciones en los componentes con el método `hasAnyAuthority()` del servicio `AccountService` .

6. Integración de la API de letras

6.1 API seleccionada: lyrics.ovh

Se eligió `lyrics.ovh` por:

- **Gratuita y sin clave de API:** no requiere registro ni límites de uso que afecten al TFG.
- **CORS habilitado:** permite peticiones directas desde el navegador sin necesidad de un proxy en el backend.
- **Simple:** endpoint único `GET https://api.lyrics.ovh/v1/{artista}/{titulo}` .

6.2 Implementación

El servicio `LyricsService` encapsula la petición HTTP externa. El componente de la barra del reproductor inyecta este servicio y gestiona los estados de carga, éxito y error mediante señales Angular. El panel se muestra/oculta con un botón en el área derecha de la barra del reproductor.

7. Problemas encontrados y soluciones

7.1 Integración de roles personalizados

JHipster genera por defecto los roles `ROLE_USER` y `ROLE_ADMIN`. Para añadir `ROLE_EDITOR` y `ROLE_ARTIST` fue necesario:

- Añadir los nuevos roles en la tabla `authority` mediante una migración Liquibase.
- Registrarlos como constantes en `AuthoritiesConstants.java`.
- Adaptar los guards y las comprobaciones de roles en el frontend.

7.2 Diseño responsive de la sidebar y el player

La coexistencia de sidebar fija, navbar superior y player bar inferior requirió un sistema de márgenes y z-index coordinados en el SCSS global para evitar solapamientos en distintos tamaños de pantalla.

7.3 Estado del reproductor sin backend de audio

El reproductor visual está implementado pero la reproducción de audio real depende de que las canciones tengan una URL de archivo válida en el campo `fileUrl`. Para la demo se utilizan estados visuales con señales Angular.

8. Conclusiones y trabajo futuro

8.1 Conclusiones

El proyecto ha permitido aplicar en un caso real los conocimientos adquiridos durante el grado: arquitectura cliente-servidor, patrones de diseño de interfaces, seguridad basada en tokens y consumo de APIs externas. La elección de Angular 21 con señales y componentes standalone ha resultado en un código más conciso y predecible comparado con versiones anteriores del framework.

8.2 Trabajo futuro

- Reproducción de audio real integrando un servicio de almacenamiento de ficheros (S3, Cloudinary).
- Letras sincronizadas con la reproducción (formato LRC) mediante `lrc1ib.net`.
- Sistema de recomendación de canciones basado en el historial de reproducciones.
- Aplicación móvil nativa con el mismo backend (Ionic o React Native).
- Despliegue en producción con CI/CD automatizado.